Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Algoritma özelliklerinin kısa tekrarı
  - Bir algoritma açık (clear) olmalıdır: bir adımı herkes aynı biçimde anlamalıdır.
  - Sonlu (finite) olmalıdır: belli bir süre sonra mutlaka bitmelidir.
  - Etkin (effective) olmalıdır: kullanılamayacak kadar verimsiz bir yöntem pratikte algoritma sayılmaz.
- Çalışma zamanı analizi için hesaplama modeli
  - Bilgisayar otomat, Turing makinesi ya da RAM (Random Access Machine) modeliyle gösterilebilir. Derste bugünkü bilgisayarlara benzeyen RAM modeli kullanılır.
  - Toplama, çarpma, bölme, atama gibi her temel işlemin birim zaman (unit time) aldığı ve her kelimenin (word) aynı boyutta olduğu varsayılır.
  - Gerçek mimaride bazı komutlar birden çok döngüde (cycle) çalışsa da analizde karmaşık işlemler temel işlemlere ayrılarak sayılır.
- İki algoritmayı karşılaştırma sorunu
  - Slayttaki örnek: $T_A(N) = 1000N$, $T_B(N) = N^2$ (ekrandan, 00:05:20). Hangisinin hızlı olduğu girdi boyutuna bağlıdır.
  - $N < 1000$ için B, $N > 1000$ için A daha hızlıdır; $N = 1000$'de ikisi eşittir.
  - Gerçek çalışma süresi derleyiciye, işletim sistemine ve bilgisayara bağlıdır. Bunlar genelde sabitleri ve düşük dereceli terimleri etkiler; algoritmanın kendisi için asimptotik notasyon kullanılır (ekrandan, 00:10:00).
- Çalışma zamanı fonksiyonu (running time function)
  - Pozitif tam sayılardan (girdi boyutu) gerçel sayılara giden, azalmayan (non-decreasing) ve yeterince büyük n için pozitif değer alan fonksiyondur.
- Big O notasyonu (üst sınır)
  - Tanım (ekrandan, 00:13:00): $T(N) = O(f(N))$ ancak ve ancak pozitif c ve n_0 sabitleri için $N \geq n_0$ iken $T(N) \leq c\, f(N)$ ise.
  - $T(N)$, $f(N)$'den daha hızlı büyümez. n_0'dan küçük değerler önemsenmez.
  - Örnekler (ekrandan, 00:20:40): $7n + 5 = O(n)$ ($c = 8$, $n_0 = 5$); $7n + 5 = O(n^2)$ ($c = 7$, $n_0 = 2$); $7n^2 + 3n = O(n)$ değildir, çünkü sabit bir c bulunamaz.
- Omega notasyonu (alt sınır)
  - $T(N) = \Omega(f(N))$ ancak ve ancak $N \geq n_0$ iken $T(N) \geq c\, f(N)$ ise. Algoritma bu sınırdan daha hızlı çalışmaz.
  - Slayttaki örnekler (ekrandan, 00:50:00): $3n+2 = \Omega(n)$, $6 \cdot 2^n + n^2 = \Omega(2^n)$, $3n-7 = \Omega(1)$.
- Theta notasyonu (sıkı sınır)
  - Aynı fonksiyon hem üst hem alt sınır olarak yazılabiliyorsa $T(N) = \Theta(h(N))$ denir.
  - Örnek (ekrandan, 01:00:00): $7n^2 + 3n + 5 = O(n^2) = \Omega(n^2) = \Theta(n^2)$.
  - $f(n) = O(g(n))$ ancak ve ancak $g(n) = \Omega(f(n))$. Theta bir denklik bağıntısıdır.
- Little o notasyonu
  - Sıkı olmayan üst sınırdır: $T(N) = o(p(N))$ ise $T(N) = O(p(N))$'dir ama $\Theta(p(N))$ değildir.
  - Örnek: $3n^2 = o(n^3)$ ve $o(n^4)$'tür ama $o(n^2)$ değildir.
- Kurallar
  - Kural 1 (ekrandan, 01:18:10): $T_1 = O(f)$, $T_2 = O(g)$ ise $T_1 + T_2 = \max(O(f), O(g))$ ve $T_1 \cdot T_2 = O(f \cdot g)$.

  - Kural 2 (ekrandan, 01:20:00): $T(N)$ k. dereceden polinom ise $T(N) = \Theta(N^k)$.
  - Her k için $\log^k N = o(N)$: logaritma, N'nin her kuvvetinden yavaş büyür.
- Yaygın fonksiyonların büyüme sırası (ekrandan, 01:23:30)
  - $c = o(\log N)$, $\log N = o(\log^2 N)$, $\log^2 N = o(N)$, $N = o(N \log N)$, $N = o(N^2)$, $N^2 = o(N^3)$, $N^3 = o(2^N)$.
- Limit ile karşılaştırma
  - $\lim_{n\to\infty} f(n)/g(n) = 0$ ise $f = o(g)$; sabit $c \neq 0$ ise $f = \Theta(g)$; ∞ ise $g = o(f)$; limit yoksa iki fonksiyon arasında bu bağıntılardan biri kurulamaz.
- Maksimum alt dizi toplamı problemi (Maximum Subsequence Sum)
  - Ardışık elemanlardan oluşan ve toplamı en büyük alt diziyi bulma problemidir. Slayttaki dizi: $-2, 11, -4, 13, -5, -2$; $A[2..4]$ toplamı 20, $A[2..5]$ toplamı 15 (ekrandan, 01:29:20).
  - N^3, N^2, $N \log N$ ve N zamanlı çözümleri vardır. Küçük N için hepsi hızlıdır; büyük N için doğrusal olan en iyisidir (ekrandan, 01:30:00).
  - Kaba kuvvet (brute force) çözüm üç iç içe döngüyle N^3'tür. Önceki toplamı bir değişkende tutup üçüncü döngüden kurtulunca N^2 olur.
- Döngülerin ve if-else yapısının asimptotik analizi
  - Tek döngü $\Theta(n)$, iç içe iki döngü $\Theta(m \cdot n)$ ya da $\Theta(R \cdot Q)$.
  - Kural 3, ardışık deyimler (ekrandan, 01:35:20): süreler toplanır. $\Theta(N)$ döngüyü $\Theta(N^2)$ iç içe döngü izliyorsa toplam $\Theta(N^2)$'dir.
  - if-else: koşul süresi T_3 her durumda eklenir. En kötü durumda $\max(T_1, T_2)$, en iyi durumda $\min(T_1, T_2)$, ortalamada olasılıklarla ağırlıklı toplam alınır.
- En kötü, ortalama ve en iyi durum (ekrandan, 01:40:00)
  - $T_{worst}(N) = \max_{|I|=N} T(I)$ (genelde kullanılan), $T_{av}(N) = \sum_{|I|=N} T(I) \Pr(I)$ (hesabı zor), $T_{best}(N) = \min_{|I|=N} T(I)$.
  - $T_{worst} \geq T_{av} \geq T_{best}$ ve $T(n) = O(T_{worst}(n)) = \Omega(T_{best}(n))$.
  - Doğrusal arama: en kötü n, eşit olasılıkla ortalama yaklaşık $n/2$, en iyi 1.
- Analizi deneyle kontrol etme (ekrandan, 01:50:00)
  - $T_e(N)/f(N)$ oranı hesaplanır, N katlanarak artırılır. Oran sabite yakınsıyorsa $f(N)$ sıkı sınırdır; sıfıra yakınsıyorsa üst sınır gereğinden büyük seçilmiştir (overestimate); ıraksıyorsa süre olduğundan küçük tahmin edilmiştir (underestimate).
- Algoritma karmaşıklığı ile problem karmaşıklığı
  - Algoritma karmaşıklığı belli bir algoritmanın süresini, problem karmaşıklığı ise problemi çözebilecek en iyi algoritmanın süresini anlatır.
  - Doğrusal aramada her elemana bir kez bakmak gerektiği için alt sınır n'dir.
- Özyinelemeli (recursive) çağrıların analizi
  - Faktöriyel (ekrandan, 02:00:00): `if (n<=1) return 1; else return n*factorial(n-1);`. Bir çarpma ve bir çıkarma olduğu için $T(n) = 2 + T(n-1)$, $T(1) = 1$.
  - İleri yerine koyma (forward substitution): başlangıç koşulundan başlayıp birkaç terim yazılır, örüntü tahmin edilir, kapalı form bulunur, sonra tümevarımla ispatlanır.
  - Geri yerine koyma (backward substitution): $T(n)$'den başlayıp $T(n-1)$, $T(n-2)$, ... yerine konur, genel k (ya da i) terimi yazılır, başlangıç koşuluna ulaşan k seçilir.
  - Faktöriyel sonucu: $T(n) = 2(n-1) + 1 = 2n - 1 = \Theta(n)$ (ekrandan, 02:10:00).
  - Fibonacci: $T(N) = T(N-1) + T(N-2) + 2$, slaytta $O(2^n)$ (ekrandan, 02:14:40). Hoca iki çıkarma ve bir toplama olduğu için $+3$ yazmanın daha doğru olacağını söyledi.
  - Hanoi kuleleri: $T(n) = 2T(n-1) + 1$, $T(1) = 1$; çözüm $2^n - 1 = \Theta(2^n)$ (ekrandan, 02:33:00).
  - $T(n) = 2T(\sqrt{n}) + 1$, $T(2) = 0$: $i = \log\log n$ seçilir, sonuç $\Theta(\log n)$ (ekrandan, 02:40:00).

Hocanın Özellikle Vurguladığı Kısımlar

- Hangi algoritmanın daha hızlı olduğu girdi boyutu bilinmeden söylenemez
  - $1000N$ ile N^2 örneğinde küçük girdilerde N^2 daha hızlıdır. Bu yüzden karşılaştırma büyük N davranışına, yani asimptotik notasyona dayanır.
- Sıkı (tight) sınır aranır

  - $1000n = O(n^2)$ ya da $O(n^4)$ yazmak yanlış değildir, ama bilgi vermez. Hoca bunu "beş yıl içinde olacak" ile "bir ay içinde olacak" sözlerinin farkına benzetti.
  - Aynı şekilde $\Omega(1)$ hemen her fonksiyonun alt sınırıdır; anlamlı alt sınır fonksiyona yakın olandır.
- Theta en faydalı bilgiyi verir
  - Theta hem üst hem alt sınırın aynı olduğunu söyler. Bazı algoritmalarda (karmaşık polinomlar, girdi dağılımına bağlı koşullar) Theta bulunamaz; o zaman Big O ve Omega kullanılır. Bu dersteki algoritmaların çoğunda Theta bulunabilecektir.
- Yazım stili
  - $4n^2$ için $O(2n^2)$ ya da $O(n^2 + n)$ yazmak notasyon olarak doğru ama kötü stildir. Sabitler ve düşük dereceli terimler atılır: $O(n^2)$ ya da $\Theta(n^2)$.
- if-else'de olasılık
  - Olasılıklar biliniyorsa önce dallar olasılıklarıyla çarpılmalı, sonra en büyüğü alınmalıdır. Doğrudan büyük dal alınırsa, çok nadir girilen pahalı bir dal yüzünden gereğinden yüksek bir üst sınır bulunabilir.
- Hazır algoritma seçerken problem incelenmeli
  - Kütüphanelerdeki algoritmalar genelde daha verimli uygulanmıştır, sıfırdan yazmaya gerek yoktur. Ama ilk bulunan alınmamalı: ortalama durumda n, en kötü durumda n^3 çalışan bir algoritma, girdiler hep kötü duruma düşüyorsa yanlış seçimdir.
  - Yeni bir algoritma yazılıyorsa önce teorik analiz, sonra uygulama, sonra deneysel kontrol yapılır.
- Slayttaki hatalı grafik
  - Bir grafikteki azalan mavi eğri çalışma zamanı olamaz, çünkü çalışma zamanı fonksiyonu azalmayan bir fonksiyondur.
- Özyinelemede amaç T'li terimden kurtulmaktır
  - Genel terim yazıldıktan sonra i ya da k, başlangıç koşuluna ulaşacak biçimde n cinsinden seçilir. Kök içeren yinelemelerde bu seçim genelde logaritmalı olur.

Kısa Tekrar Notları

- RAM modelinde her temel işlem birim zamandır.
- O: üst sınır, Ω: alt sınır, Θ: ikisi aynı, o: sıkı olmayan üst sınır.
- O tanımı: $N \geq n_0$ için $T(N) \leq cf(N)$. Ω için $\geq$.
- Sabit katsayılar ve düşük dereceli terimler atılır; k. derece polinom $\Theta(N^k)$'dir.
- Toplamda büyük terim, çarpımda çarpım kalır.
- Büyüme sırası: $c < \log N < \log^2 N < N < N \log N < N^2 < N^3 < 2^N$.
- Limit 0 ise $f = o(g)$, sabit ise Θ, sonsuz ise $g = o(f)$.
- Maksimum alt dizi toplamı: N^3, N^2, $N \log N$, N çözümleri var; örnek dizide cevap 20.
- Ardışık bloklar toplanır, iç içe döngüler çarpılır, if-else'de en kötü durumda büyük dal alınır.
- $T_{worst} \geq T_{av} \geq T_{best}$; $T = O(T_{worst}) = \Omega(T_{best})$.
- Deneysel kontrol: T_e/f sabite gidiyorsa sınır sıkı, sıfıra gidiyorsa fazla tahmin, ıraksıyorsa eksik tahmin.
- Faktöriyel $\Theta(n)$, Hanoi $\Theta(2^n)$, $2T(\sqrt{n}) + 1$ yinelemesi $\Theta(\log n)$.
- İleri ve geri yerine koymada bulunan formül tümevarımla ispatlanmalıdır.

Detaylı Açıklamalar

- **Neden asimptotik notasyon?** Table yöntemiyle (her satırın kaç kez çalıştığını sayma) analiz edilen büyük bir kod, $4n^3 + 2n^2 + 189$ gibi çok terimli bir polinom verir (ekrandan, 00:40:00). İçinde if'ler varsa olasılıklar, bazen logaritmalı terimler de girer. İki algoritmayı bu polinomlarla karşılaştırmak anlamlı değildir. Bunun yerine sabitler ve düşük dereceli terimler atılır, $4n^3 + 2n^2 + 189 = O(n^3)$ yazılır. Bu, algoritmanın n^3'ten daha hızlı büyümediğini söyler. Ayrıca makineye bağlı farklar da çoğunlukla sabitlere yansıdığı için notasyon makineden bağımsız bir karşılaştırma sağlar.
- **Big O'yu sınamak.** Bir ifadenin $O(f)$ olduğunu göstermek için uygun c ve n_0 aranır. $7n + 5 \leq 8n$ eşitsizliği $n \geq 5$ için sağlanır; bu yüzden $7n + 5 = O(n)$. $7n + 5 \leq 7n^2$

eşitsizliği $n \geq 2$ için sağlanır ($n = 2$'de $19 \leq 28$); bu yüzden $7n+5 = O(n^2)$ da doğrudur. $7n^2 + 3n \leq cn$ için hoca önce $n = 1$ koydu ve $10 \leq c$ buldu. $c = 10$ alınca $n = 2$'de $34 \leq 20$ sağlanmaz. n büyüdükçe sol taraf n^2 ile büyüdüğü için hiçbir sabit c yetmez, yani $7n^2 + 3n \neq O(n)$. Bir öğrencinin sorusu üzerine: $4n^3 = O(n^3)$ için $c = 4$ yeter; polinomda başka terimler varsa c onları da karşılayacak kadar büyük seçilmelidir.

- **Üst sınırların zinciri.** $1000n \leq cn$ ise $cn \leq cn^2 \leq cn^3 \leq \ldots$ olduğundan $1000n$ aynı zamanda $O(n^2)$, $O(n^3)$, ... olur. Bunların hepsi doğrudur ama sıkı değildir. Algoritmayı tanımlarken en sıkı üst sınır verilir. Aynı fikir alt sınır için de geçerlidir: $7n^2 + 3n + 5$ için $\Omega(n^2)$, $\Omega(n)$ ve $\Omega(1)$ doğrudur (ekrandan, 01:00:00). Bir öğrenci $\Omega(n)$'nin nasıl yazıldığını sordu; hoca $c = 8$ alıp $n = 1$'de $15 \geq 8$, $n = 2$'de ve sonrasında da eşitsizliğin sağlandığını gösterdi.
- **Theta ve O/Omega ilişkisi.** $f(n) = O(g(n))$, yani $f(n) \leq c_1 g(n)$ ise iki taraf c_1'e bölünerek $g(n) \geq \frac{1}{c_1} f(n)$ elde edilir. $c_2 = 1/c_1$ seçilince $g(n) = \Omega(f(n))$ olur. $f(n) = 2n^2$ için hem $2n^2 \leq cn^2$ hem $2n^2 \geq cn^2$ $c = 2$ ile sağlanır; bu yüzden $2n^2 = \Theta(n^2)$. Örnekler: $3n+2 = \Theta(n)$, $10 \log n + 4 = \Theta(\log n)$; $3n+2 = \Theta(1)$ değildir, çünkü 1 yalnız alt sınırdır.
- **Little o ve limit.** Little o, Big O'nun Theta olmayan durumudur. $3n^2$ için $o(n^3)$ ve $o(n^4)$ doğru, $o(n^2)$ yanlıştır. Limit yöntemi bu bağıntıları hızlıca verir: f/g oranı sıfıra gidiyorsa g çok daha hızlı büyüyordur ($f = o(g)$); sabite gidiyorsa ikisi aynı hızda büyür ($f = \Theta(g)$ ve $g = \Theta(f)$); sonsuza gidiyorsa f daha hızlıdır ($g = o(f)$). Limit bu üçünden biri değilse iki fonksiyon sürekli birbirini geçiyordur ve aralarında bu bağıntılardan biri yoktur. $\log^k n$'deki k üsttür, taban 2 kabul edilir: $\log^2 n = \log n \cdot \log n$.
- **Kural 1'in sezgisi.** İki polinom toplanırken aynı dereceli terimlerin katsayıları toplanır, en yüksek derece değişmez; bu yüzden toplam büyük olanın mertebesindedir. Çarpımda üsler toplanır; bu yüzden mertebe $f \cdot g$ olur. Kural 2'nin ispatında her katsayı en büyük katsayı a ile değiştirilerek polinom üstten sınırlanır.
- **Maksimum alt dizi toplamı.** Dizi $-2, 11, -4, 13, -5, -2$ iken $-2+11 = 9$, $11-4+13 = 20$, $11 - 4 + 13 - 5 = 15$ olur; en büyük toplam 20'dir ($A[2..4]$). Kaba kuvvet çözümde başlangıç i, bitiş j ve i'den j'ye toplama için üç döngü vardır: N^3. j bir artınca toplamı baştan hesaplamak yerine önceki toplama yeni elemanı eklemek bir döngüyü kaldırır: N^2. Hoca $N \log N$ çözümün parçaları birleştirerek ilerlediğini söyledi ve ayrıntısını sonraki derslere bıraktı. Küçük N'de fark görünmeyebilir, ama büyük N'de $N \log N$ ile N^3 arasında "dağlar kadar fark" oluşur.
- **if-else örneği.** Koşul $\Theta(n)$, if dalı $\Theta(n^2)$, else dalı $\Theta(n)$ olsun. En kötü durum $T_3 + \max(T_1, T_2) = \Theta(n^2)$, en iyi durum $T_3 + \min(T_1, T_2) = \Theta(n)$. Olasılıklar $1/2$ ise ortalama $\frac{1}{2}n + \frac{1}{2}n^2$ terimleriyle yine $\Theta(n^2)$'dir. Buna göre algoritma için $O(n^2)$ ve $\Omega(n)$ yazılabilir.
- **Deneysel kontrol.** Rastgele girdiler $N, 2N, 4N, \ldots$ boyutlarında üretilir ve gerçek süre ölçülür. Süre birimi (örneğin nanosaniye) önemli değildir; bakılan şey girdi iki katına çıkınca sürenin ne kadar arttığıdır. Süre teorik tahminden yavaş artıyorsa sınır fazla yüksek bulunmuştur; çok daha hızlı artıyorsa olasılıklar yanlış hesaplanmış ya da atılan terimler büyük N'de önem kazanmış olabilir.
- **Özyinelemeli algoritmaların analiz adımları.** Önce girdi boyutunu gösteren parametre ve temel işlem belirlenir. Sonra fonksiyonun kendini nasıl çağırdığına bakılır (bir çıkararak mı, ikiye bölerek mi, kaç kez). Bundan yineleme bağıntısı (recurrence relation) ve durma koşullarından başlangıç koşulları yazılır. Sonra yineleme çözülür. Faktöriyelin ikinci slaytında (ekrandan, 02:20:00) yalnız çarpma sayılır: $M(n) = M(n-1) + 1$, $M(0) = 0$. İleri yerine koymayla $M(1) = 1$, $M(2) = 2$, $M(3) = 3$ bulunup $M(n) = n$ tahmin edilir; geri yerine koymayla $M(n) = M(n-i) + i$ yazılıp $i = n$ seçilir ve $M(n) = n$ bulunur. Çıkarmayı da sayan $T(n) = 2 + T(n-1)$ sürümü $2n-1$ verir; ikisi de $\Theta(n)$'dir, yani sabitin 1 ya da 2 alınması asimptotik sonucu değiştirmez.
- **Hanoi kuleleri.** Amaç n diski A'dan C'ye B'yi yardımcı kullanarak taşımaktır; büyük disk küçüğün üstüne konamaz. Özyinelemeli yaklaşım (ekrandan, 02:30:00): üstteki $n-1$ diski C yardımıyla A'dan B'ye taşı, en büyük diski A'dan C'ye taşı, $n-1$ diski A yardımıyla

B'den C'ye taşı. Buradan $T(n) = 2T(n-1)+1$, $T(1) = 1$. Geri yerine koymayla $T(n) = 2^i T(n-i) + 2^{i-1} + \cdots + 2 + 1$ bulunur; $i = n-1$ seçilince $T(n) = 2^{n-1} + 2^{n-2} + \cdots + 1 = 2^n - 1 = \Theta(2^n)$. Disk sayısı arttıkça süre üstel büyür.

- **Kök içeren yineleme.** $T(n) = 2T(\sqrt{n}) + 1$, $T(2) = 0$ için açılım $T(n) = 2^i T(n^{1/2^i}) + 2^0 + 2^1 + \cdots + 2^{i-1}$ verir. $n^{1/2^i} = 2$ olması için $i = \log\log n$ seçilir (taban 2). İlk terim $T(2) = 0$ yüzünden sıfırlanır, kalan toplam $2^{\log\log n} - 1 = \log n - 1$ olur, yani $\Theta(\log n)$. Kökten kurtulmak için logaritmalı bir i seçmek sık kullanılan bir yöntemdir; ağaç yapılarında da benzer logaritmik süreler çıkar.
- **Ders sonrası not:** Omega örneğinde hoca $3n + 2 \geq cn$ için "$n > 2$ ve $c = 4$" dedi. $3n + 2 \geq 4n$ yalnız $n \leq 2$ için doğrudur. Doğrusu $c = 3$ (ya da $c \leq 3$ olan herhangi bir pozitif sabit) ve $n_0 = 1$'dir.
- **Ders sonrası not:** Bir öğrenci Hanoi probleminin NP sınıfında olup olmadığını sordu, hoca "evet" dedi. Bu büyük olasılıkla bir karışıklıktır: Hanoi'nin süresi üsteldir, çünkü çıktının kendisi $2^n - 1$ hamledir; bu, problemi NP sınıfına sokmaz. NP, çözümü polinom zamanda doğrulanabilen karar problemleri içindir. Hoca ayrıca 15 diskle bile algoritmanın bitmeyebileceğini söyledi; 15 disk $2^{15} - 1 = 32767$ hamle eder ve hemen biter. Bitmeme sorunu ancak 100 gibi büyük disk sayılarında ortaya çıkar.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.